软件工程与方法

第7讲 敏捷方法

主要内容

1. 敏捷思想
2. 敏捷开发对软件质量的影响
3. 敏捷过程与传统方法
4. 敏捷开发方法：
   - XP
   - SCRUM



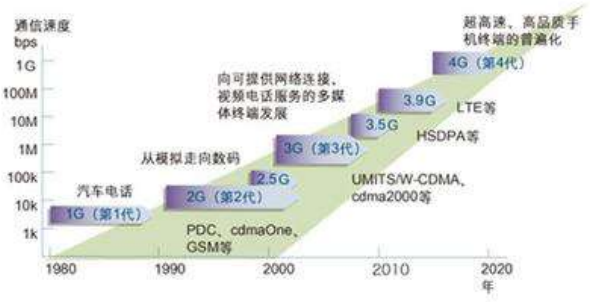
第7讲 敏捷方法

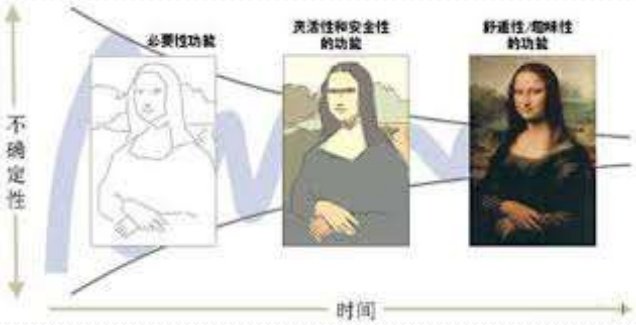
1 敏捷思想

为什么要敏捷方法？

- 当越来越多的组织要求通过及时部署基于Internet的服务来寻求获得竞争优势时，开发人员就承受不断增长的压力以尽快实现新的、增强的服务。
- 一些新的软件开发方法被提出，特别引人注目的是“**轻载**”方法和过程，它们相对于“**重载**”的方法。







IBM Rational和Yahoo敏捷转型成果

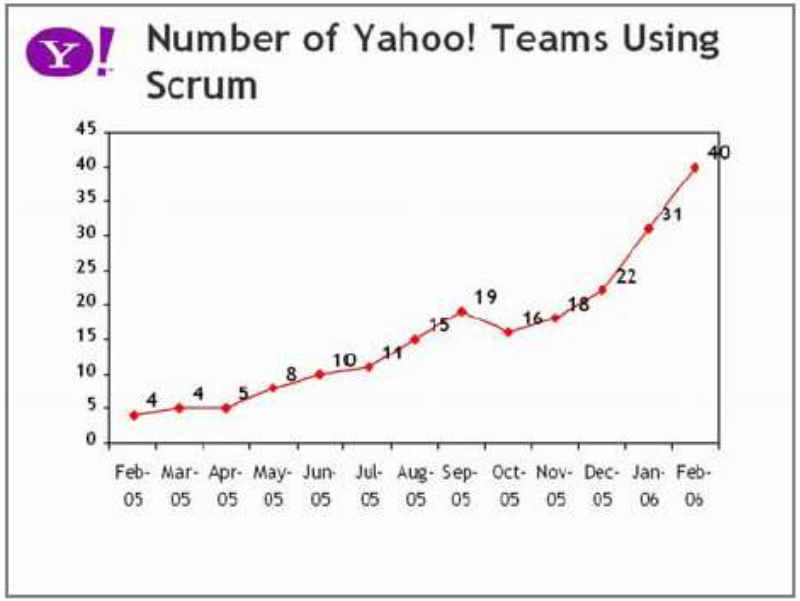


Figure 10. In one year, the number of Scrum teams rose from four to forty.

敏捷开发实践证明

- 提高了团队的开发效率
- 较低的开发成本
- 更快的上市速度
- 更加优质的软件质量
- 系统更好地适应了客户的需求
- 可重复的结果

IBM Rational敏捷转型成果截止2009年5月的成果：

- 从2007年开始，使用敏捷方法的项目年增长93%。
- 目前约50%的项目使用敏捷方法，58%的开发人员是基于敏捷方法进行开发。
- 敏捷项目的质量提高：
  - 缺陷数每年减少约13%
  - 缺陷（backlog）每年减少>12%
- 敏捷项目提高产品按时发布率>10%



敏捷方法：

- 软件开发节奏 Software Development Rhythms
- 敏捷数据库技术 AD, Agile Database Techniques
- 敏捷建模 AM, Agile Modeling
- 自适应软件开发 ASD, Adaptive Software Development
- 水晶方法 Crystal
- 特性驱动开发 FDD, Feature Driven Development
- 动态系统开发方法 DSDM，Dynamic Systems Development Method
- 精益软件开发 Lean Software Development Scrum
- 测试驱动开发 TDD, Test-Driven Development XBreed
- 极限编程 XP/eXtreme Programming
- 结对编程 Pair Programming
- 实用编程 Pragmatic Programming
- SCRUM

敏捷的理念

- 敏捷团队依靠**变化**来获取活力。
- 团队几乎**不预先设计**，因此，不需要一个成熟的初始设计。他们更愿意保持设计尽可能的干净、简单，并使用许多单元测试和验收测试作为支援。
- **设计灵活**和易于理解，利于持续地改进设计，以便每次迭代生成的系统都最适合于那次迭代的需求。
- 采取一些OO的设计原则来**避免软件“腐化”**。

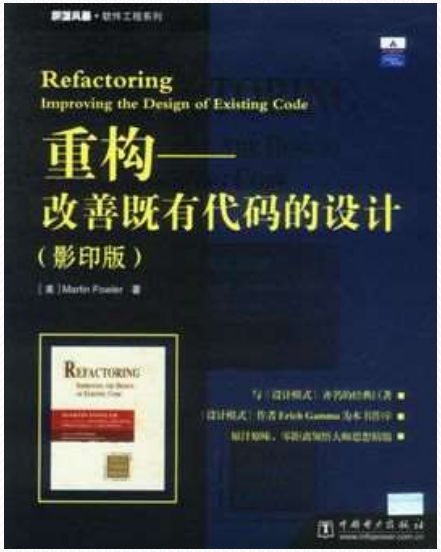
Martin Fowler: The New Methodology



13 December 2005

http://www.martinfowler.com/articles/newMethodology.html

- differences: plan-driven methodologies vs. Agile methodologies
  - Agile methods are adaptive rather than predictive.
  - Agile methods are people-oriented rather than process-oriented
- The main points of the article：
  - Predictive versus Adaptive
    - Separation of Design and Construction
    - The Unpredictability of Requirements
    - Is Predictability Impossible?
    - Controlling an Unpredictable Process – Iterations
    - The Adaptive Customer
  - Putting People First
    - Plug-Compatible Programming Units
    - Programmers are Responsible Professionals
    - Managing a People Oriented Process
    - The Difficulty of Measurement
    - The Role of Business Leadership
  - The Self-Adaptive Process



敏捷宣言

Manifesto for Agile Software Development stated:

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactive over processes and tools

Working software over comprehensive documentation

Customer collaboration over contact negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

2001年，17位业界专家，在软件过程未来发展趋势研讨会上，就什么是“敏捷”达成共识，并成立了敏捷联盟(Agile Alliance)，发表了敏捷宣言(Manifesto for Agile Software Development) (https://www.agilealliance.org/agile101/the-agile-manifesto/) 。

敏捷联盟定义的“敏捷原则”

1. 我们最优先要做的是通过尽早、持续地交付有价值的软件来使客户满意。
2. 在项目的整个开发期间，业务人员和开发人员必须天天在一起工作。
3. 即使到了开发后期，也欢迎需求变化。
4. 经常性地交付可以工作的软件。
5. 可以工作的软件是主要的进度度量标准。
6. 围绕被激励起的个体来构建项目。为他们提供所需的环境和支持，并信任他们能胜任工作。
7. 最好的架构、需求和设计来自于自组织的团队。
8. 在团队内部，最有效果和最有效率的传递信息的方法是面对面地交流。
9. 敏捷过程提倡可持续的开发速度。
10. 不断地关注最优秀的技术和良好的设计能增强敏捷能力。
11. 简单是根本的。
12. 开发团队每隔一定时间，都会对如何能有效地工作进行反省，然后相应地对自己的行为进行调整。

敏捷过程的11个假设

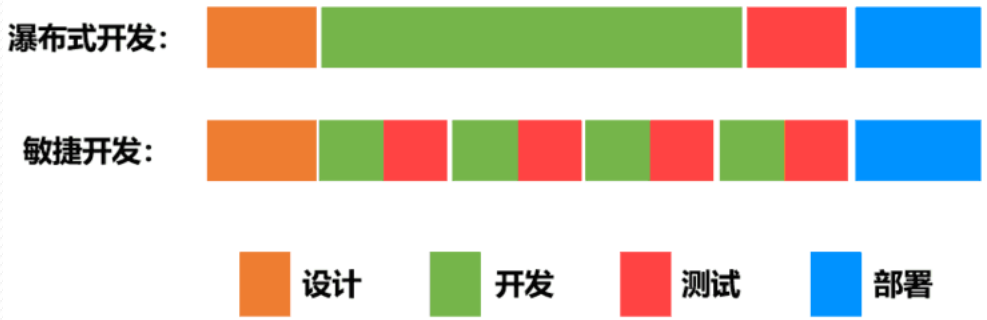
1. 客户要和开发团队协同工作，随时作好和开发人员交流的准备。而且，面对面的交流需要开发人员彼此位于很近的位置。
2. 文档和软件模型在软件开发中不充当重要的角色。
3. 软件需求和软件开发环境随着软件开发的发展而发展。
4. 动态适应不断变化的项目和产品特征的开发过程，更有可能开发出高质量的产品。
5. 开发人员具有正确地定义和适应过程的经验。换句话说，一个组织能建立由有丰富经验的问题解决者组成的团队，他们在执行过程期间，能有效地改进过程。
6. 项目的可见性能主要通过增量和一些度量的传递来获取。
7. 软件制品（产品和过程）严格的评估仅限于经常非正式的审查和代码测试。
8. 重用性和通用性不应该是面向应用程序软件开发过程的目标。
9. 变更的成本不随着时间的变化而显著增加。
10. 软件可以被增量开发。
11. 无需为变更作设计，因为任何变更能通过重构代码有效地处理。

什么是敏捷？

- 敏捷是一套价值观和一组相关的用以管理和交付项目的技术与方法的集合。敏捷强调迭代和增量开发、与客户密切合作以及跨职能团队，以构建可工作的软件。







敏捷方法的定义

[Lindvall M , Basili V , Boehm B , et al. Empirical Findings in Agile Methods[M]// Extreme Programming and Agile Methods — XP/Agile Universe 2002. Springer Berlin Heidelberg, 2002.]

Agile Methods are:

- Iterative
- Incremental
- Self-organizing (The team has the autonomy to organize itself to best complete the work items.)
- Emergent (Technology and requirements are “allowed” to emerge through the product development cycle.)

All Agile methods follow the four values and 12 principles of the Agile Manifesto.

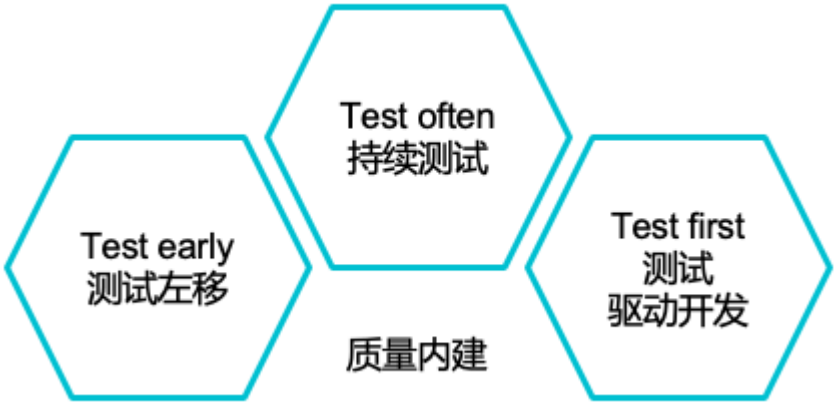
敏捷开发（Agile Development）

- 敏捷开发以用户的需求进化为核心，采用迭代、循序渐进的方法进行软件开发。
- 在敏捷开发中，软件项目在构建初期被切分成多个子项目，各个子项目的成果都经过测试，具备可视、可集成和可运行使用的特征。换言之，就是把一个大项目分为多个相互联系，但也可独立运行的小项目，并分别完成，在此过程中软件一直处于可使用状态。

来自：https://baike.baidu.com/item/敏捷开发/5618867?fr=aladdin

敏捷测试

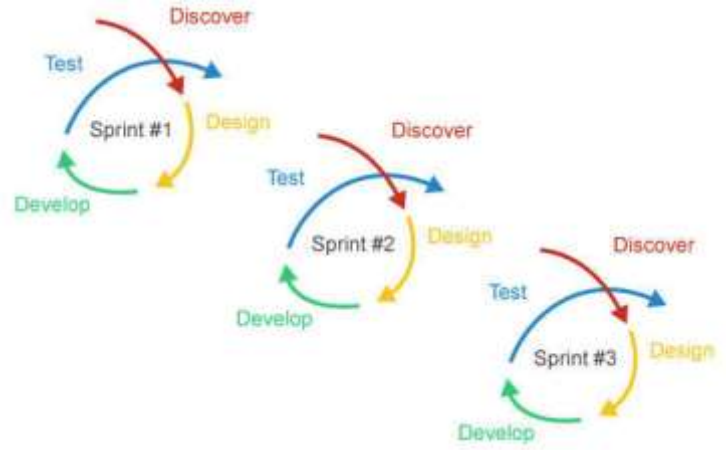
- 敏捷测试是伴随着敏捷开发过程的所有质量相关活动，有着如下的特点：
  - 不能独立存在，不是一种测试类型或方法
  - 敏捷测试不仅是测试人员的工作，敏捷测试是团队的活动
  - 抛开敏捷开发谈敏捷测试没有意义
- 敏捷测试的目标也不再是发现更多的bug，而是尽快的交付高质量的软件。

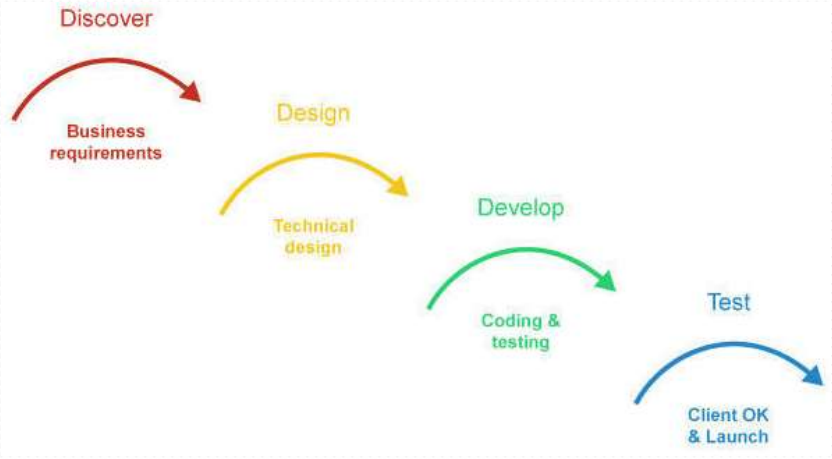


1. 敏捷测试的核心是质量内建，而质量内建就是缺陷预防；
2. 测试左移、全阶段的持续测试、测试驱动开发是质量内建成功的关键

敏捷的特点总结

- 敏捷方法适应了“网络时代”软件开发。
- 敏捷开发是一个过程，不是一个事件。它是一些原则、模式以及实践的总结。
- 敏捷过程致力于保持系统设计在任何时间都尽可能得简单、干净和富有表现力。
- 敏捷开发以用户的需求进化为核心，采用迭代、循序渐进的方法进行软件开发。





第7讲 敏捷方法

2 敏捷建模

敏捷建模的核心思想

① 迭代和增量建模
- 使用合适的制品
- 并行创建多个模型
- 迭代到其他的制品中
- 小增量建模

② 团队协作
- 与他人一起建模
- 项目干系人的积极参与
- 集体所有
- 公开展示模型

③ 简单性
- 保持简单的内容
- 简单地描述模型
- 使用最简单的工具

④ 验证
- 考虑可测试性
- 用代码验证

敏捷开发的面向对象设计原则：

- 单一职责原则(SRP)

  就一个类而言，应该仅有一个引起它变化的原因。

- 开放-封闭原则(OCP)

  软件实体应该是可以扩展的，但是不可修改。

- Liskov替换原则(LSP)

  子类型必须能够替换掉它们的基类型。

- 依赖倒置原则(DIP)

  抽象不应该依赖于细节。细节应该依赖于抽象。

- 接口隔离原则(ISP)

  不应该强迫客户依赖于它们不用的方法。接口属于客户，不属于它所在的类层次结构。

- 重用发布等价原则(REP)

  重用的粒度就是发布的粒度。

（续）

- 共同封闭原则(CCP)

  包中的所有类对于同一类性质的变化应该是共同封闭的。一个变化若对一个包产生影响，则将对该包中的所有类产生影响，而对于其他的包不造成任何影响。

- 共同重用原则(CRP)

  一个包中的所有类应该是共同重用的。如果重用了包中的一个类，那么就要重用包中的所有类。

- 无环依赖原则(ADP)

  在包的依赖关系图中不允许存在环。

- 稳定依赖原则(SDP)

  朝着稳定的方向进行依赖。

- 稳定抽象原则(SAP)

  包的抽象程度应该和其稳定程度一致。

例：敏捷中的OOD

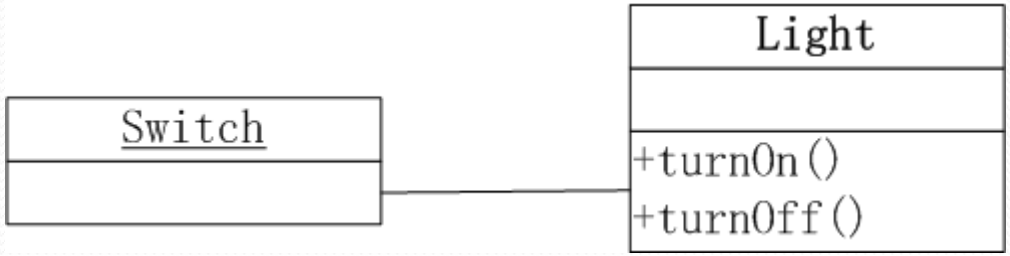
- 问题：

  选择设计运行在简易台灯中的软件，台灯由一个开关和一盏灯组成。你可以询问开关开着还是关着，也可以让灯打开或关闭。



解决方案一：

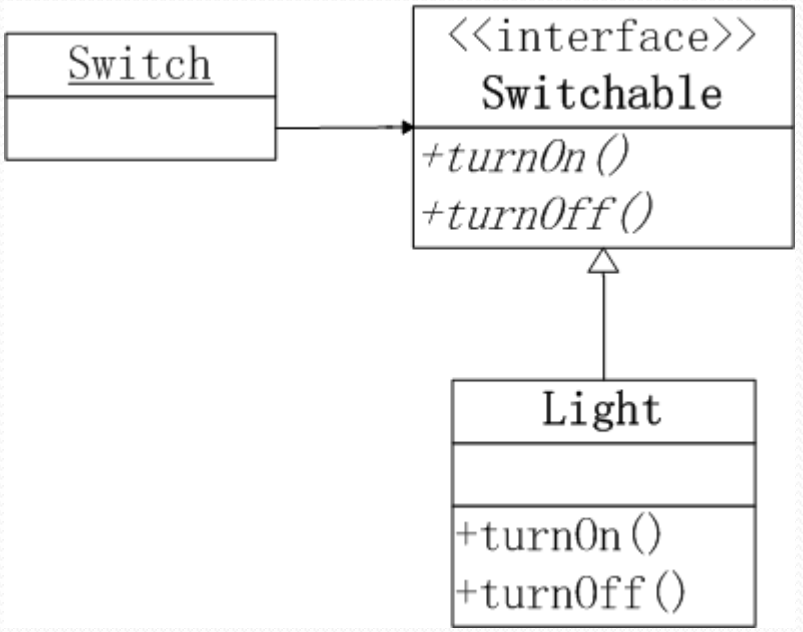
- 一种最简单的解决方案，Switch对象可以轮询真实开关的状态，并且可以发送相应的turnOn和turnOff消息给Light。



- 设计违反了两个设计原则：
  - 依赖倒置原则(DIP)
    - DIP原则要求优先依赖于抽象类
    - 而Switch依赖了具体类Light
  - 开放封闭原则(OCP)
    - 对OCP的违反是在任何需要Switch的地方都要带上Light，这样就不能容易扩展Switch去管理除Light外的其他对象。

解决方案二：

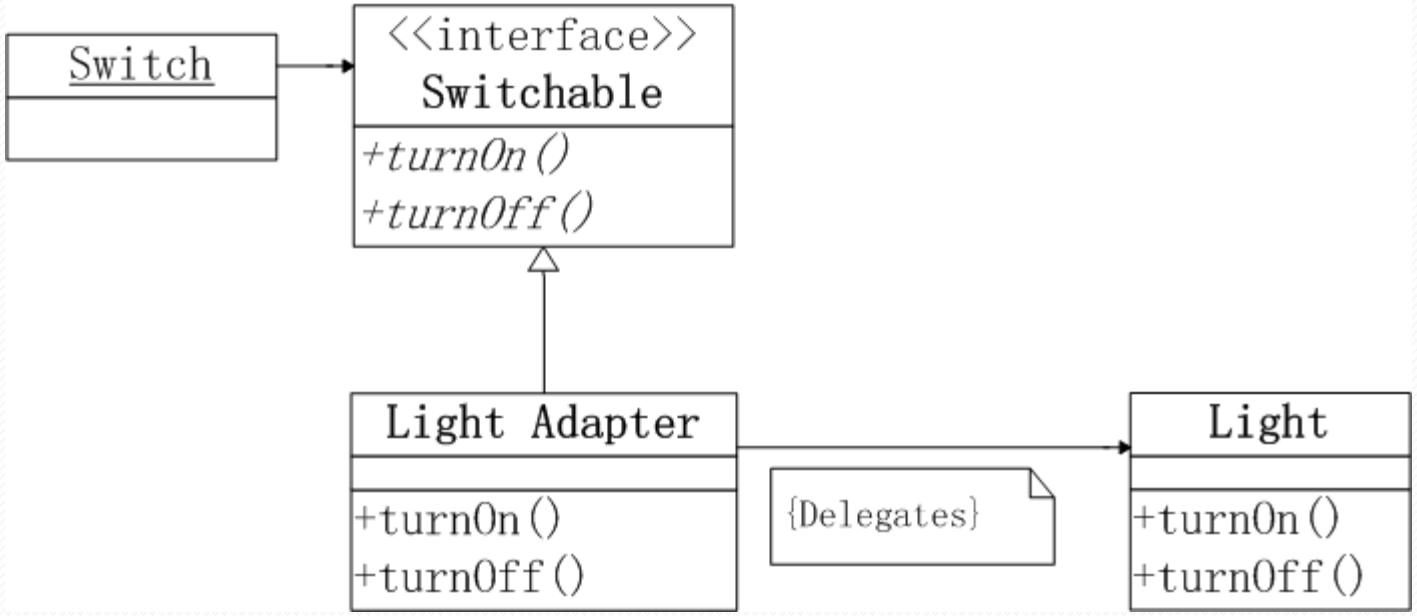
- 为了满足了DIP和OCP原则，可以采用ABSTRACT SERVER模式，在Switch和Light之间引入一个接口，这样就使得Switch能够控制任何实现了这个接口的东西。



- 方案二违返了单一职责原则(SRP)，它把Switch和Light绑定在一起，而它们可能会由于各种原因而改变。
- 这种问题可以采用ADAPTER模式来解决，适配器从Switchable 派生并委托给Light，问题就被完美的解决。

解决方案三：

- 采用ADAPTER模式，适配器从Switchable 派生并委托给Light，这样，Switch就可以控制任何能够被打开或者关闭的对象。
- 但是这也需要付出时间和空间上的代价来换取。



第7讲 敏捷方法

3. 极限编程XP

XP介绍

- XP是由Kent Beck（1999）提出的，最有特点的敏捷方法。
- 采用简单策略，集中精力在最主要的功能上，其它可以在迭代开发中不断改进，从而避免书写大量文档。
- XP采用整体价值驱动的观点，强调4个关键价值：
  - **沟通：**大多数项目的失败源于沟通不畅，所以要进行一些能够推动积极沟通的实践。
  - **简单：**开发能够满足客户需要的最简单的产品。
  - **反馈：**开发者必须要获取并且重视来自客户、系统的反馈以及相互之间的反馈。
  - **勇气：**准备好做出支持其他原则和实践的艰难决定。

XP的核心思想和特点

核心思想：

- 早期发现错误以及降低复杂度可以节约成本。
- 将任务/系统细分为可以在较短周期解决的一个个子任务/模块，并且强调测试、代码质量和及早发现问题。通常，通过一个个短小的迭代周期，获得一个个阶段性的进展，并且可以及时形成一个版本供用户参考，以便及时对用户可能的需求变更作出响应。

特点：

- 沟通反映在3个层面：客户与开发者、开发者之间以及测试者与开发者之间。
- 每一次迭代只有2周时间，目标非常简单，开发者与用户的关系也很简单。
- 你不能信任我1年，2周总可以，我2周后给你结果。如果不满意损失不大，如果成功，再信任我2周。

沟通的效率



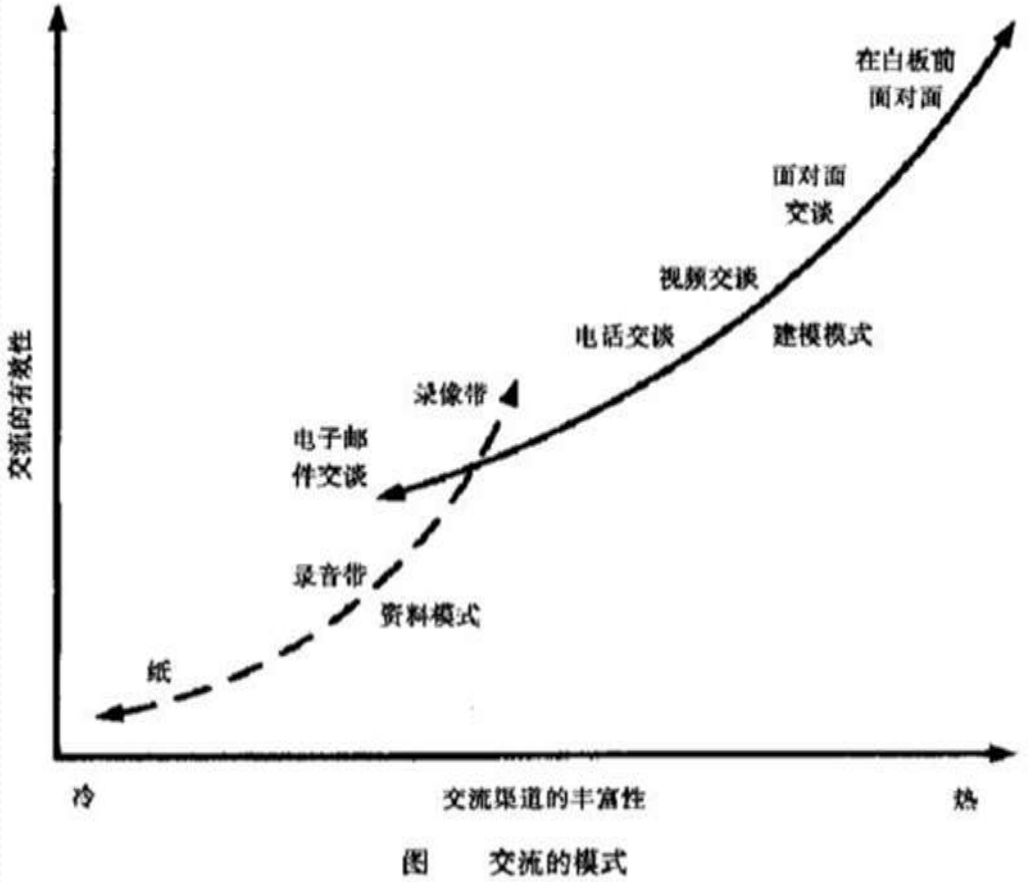


图　交流的模式

XP开发的生命周期

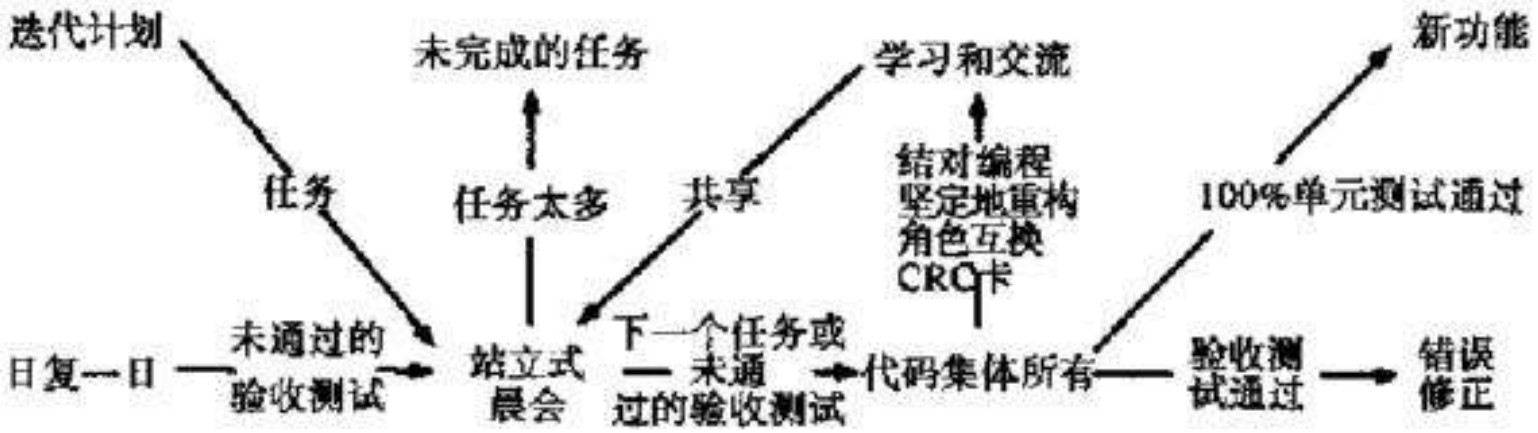


图 XP开发的生命周期 (Wells 2000)

一个XP迭代的生命周期

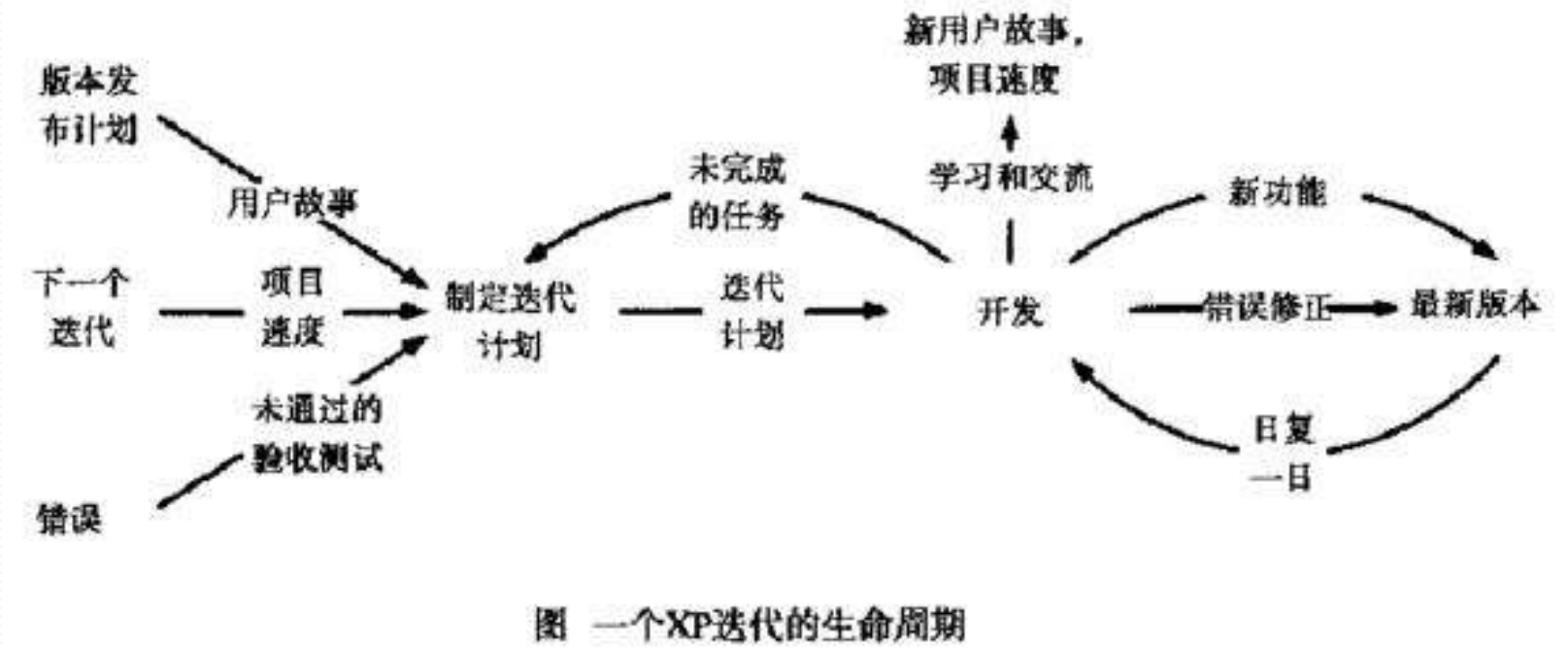


图 一个XP迭代的生命周期

XP开发人员典型的一天



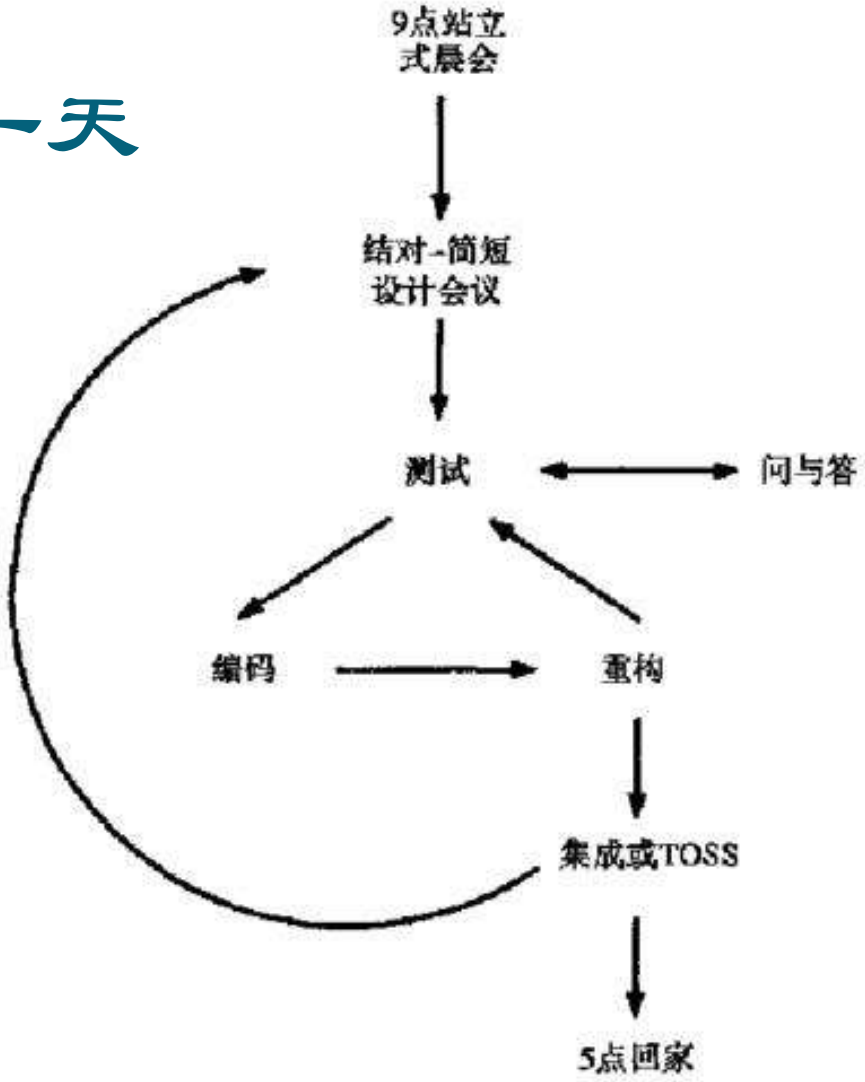


图 XP开发人员典型的一天 (Wake 2002)

XP必须遵循的12个实践：

1. **完整团队**：所有参与者（开发人员、客户、测试人员等）一起工作在一个开放的场所中，他们是同一个团队的成员。
2. **计划游戏**：计划是持续的、循序渐进的。每2周，开发人员就为下2周估算候选特性的成本，而客户则根据成本和商务价值来选择要实现的特性。
3. **客户测试**：作为选择每个所期望的特性的一部分，客户可以根据脚本语言来定义出自动验收测试来表明该特性可以工作。
4. **简单设计**：团队保持设计恰好和当前的系统功能相匹配。它通过了所有的测试，不包含任何重复，表达出了编写者想表达的所有东西，并且包含尽可能少的代码。
5. **结对编程**：所有的产品软件都是由两个程序员、并排坐在一起在同一台机器上构建的。
6. **测试驱动开发**：编写单元测试是一个验证行为，更是一个设计行为。同样，它更是一种编写文档的行为。编写单元测试避免了相当数量的反馈循环，尤其是功功能能验证方面的反馈循环。程序员以非常短的循环周期工作，他们先增加一个失败的测试，然后使之通过。

（续）

7. **改进设计**：随时利用重构方法改进已经腐化的代码，保持代码尽可能的干净、具有表达力。

8. **持续集成**：团队总是使系统完整地被集成。一个人签入（Check in）后，其它所有人负责责代码集成。

9. **集体代码所有权**：任何结对的程序员都可以在任何时候改进任何代码。没有程序员对任何一个特定的模块或技术单独负责，每个人都可以参与任何其它方面的开发。

10. **编码标准**：系统中所有的代码看起来就好像是一人单独编写的。

11. **隐喻**：将整个系统联系在一起的全局视图的概念。它是系统的未来影像，它使得所有单独模块的位置和外观变得明显直观。如果模块的外观与整个隐喻不符，那么你就知道该模块是错误的。

12. **可持续的速度**：团队只有持久才有获胜的希望。他们以能够长期维持的速度努力工作，他们保存精力，把项目看作是马拉松长跑，而不是全速短跑。

极限编程实践细则

极限编程的12个核心实践具体分来有如下四个方面一些细则内容：

1. 计划

① 写出用户素材；
② 根据发布版本计划编制进度表；
③ 频繁发布小版本:
④ 度量项目速度；
⑤ 这个项目划分为多个迭代；
⑥ 每次迭代开始前制定迭代计划；
⑦ 角色互换；
⑧ 每日立式晨会；
⑨ 根据项目情况对XP作出调整。

2. 设计

① 简单；
② 选取一个系统比喻；
③ 在设计上使用CRC卡片；
④ 通过探针方案降低风险；
⑤ 不要过早加入任何功能；
⑥ 尽可能的进行重构。

（续）

3. 编码

① 客户一直在场；
② 代码必须符合相应的编码标准；
③ 首先编写单元测试；
④ 必须实行结对编程；
⑤ 严格的串行代码集成；
⑥ 频繁地进行代码集成；
⑦ 实行代码共有制；
⑧ 将优化工作留到最后；
⑨ 不要加班。

4. 测试

① 所有代码必须有单元测试；
② 所有代码发布前必须100%通过所有对应的单元测试；
③ 发现了错误必须要增加相应的测试；
④ 经常运行接受测试并且公布其结果。

XP的限制

- XP方法不是到处可用，采用前应做评估或论证。
- 不宜运用XP方法的情况：
  ① 如果开发人员超过50人；
  ② 开发人员质量低又缺乏动力；
  ③ 客户不能理解甚至不能参与合作的情况；
  ④ 在不超过10人的场合，但场地不在一处的分布式开发，应小心。
  ⑤ 对不容易测试的应用系统，会影响XP的效果。



第7讲 敏捷方法

4. SCRUM

SCRUM

- 最初由Easel公司开发（1993），旨在寻求生产率的突破。
- SCRUM被公认为高效软件开发方法，得到非常广泛应用。
- Scrum是为迭代增量开发模型提供的一种简单流程框架，框架中包含角色、会议、规则和工件等。
- Scrum不是严格定义的流程或者方法论。
- Scrum最初是在面向对象的软件开发中开始流行,它非常适合高风险的任务。



Scrum概况

- Scrum定义：
  - Scrum是迭代式增量软件开发过程，通常用于敏捷软件开发。
  - Scrum包括了一系列实践和预定义角色的过程骨架。
- 3大角色：SM（引导者、教练和清障者）、PO、开发团队
- 4大活动：Sprint计划会、Sprint评审会、Sprint回顾会、每日立会
- 5大输出：Product Backlog、Sprint Backlog、可工作的软件、Sprint计划、燃尽图（ Burndown Chart ）

SCRUM框架

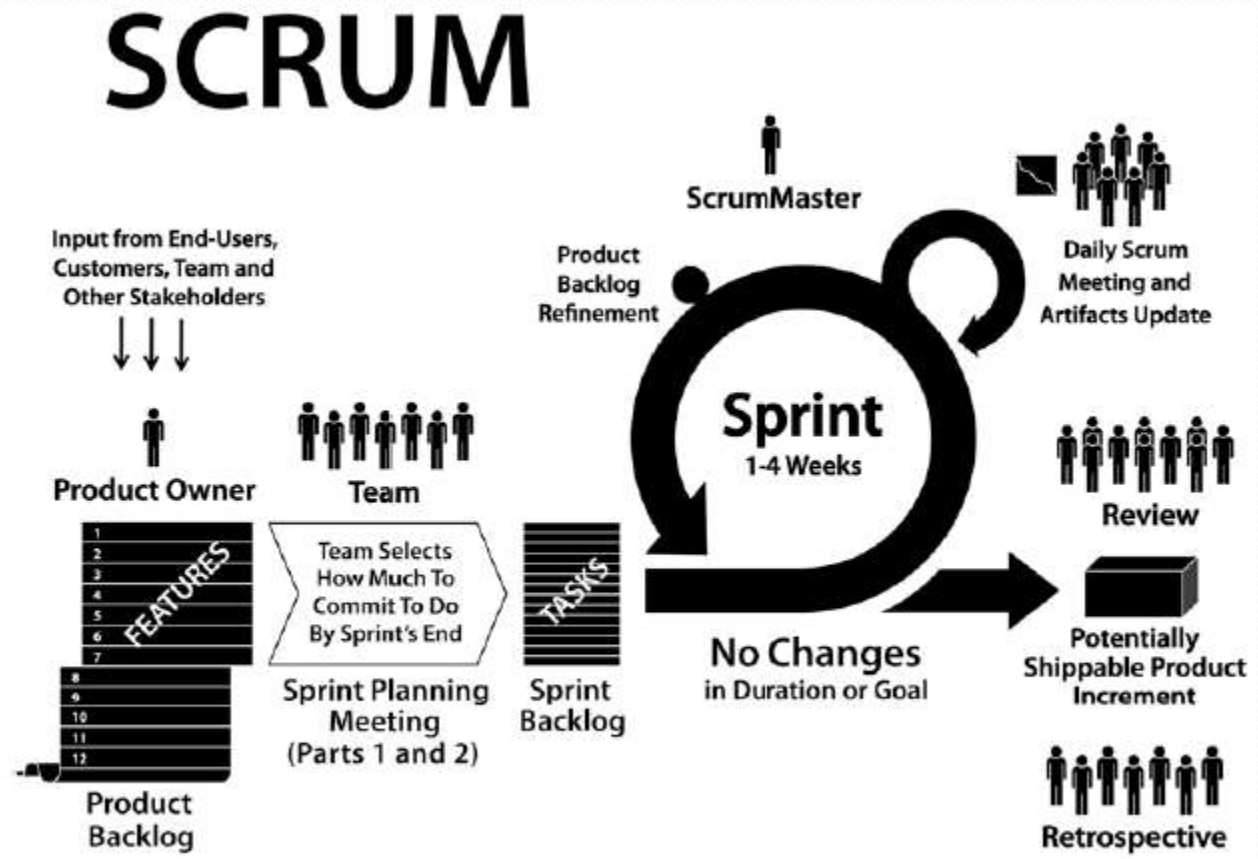
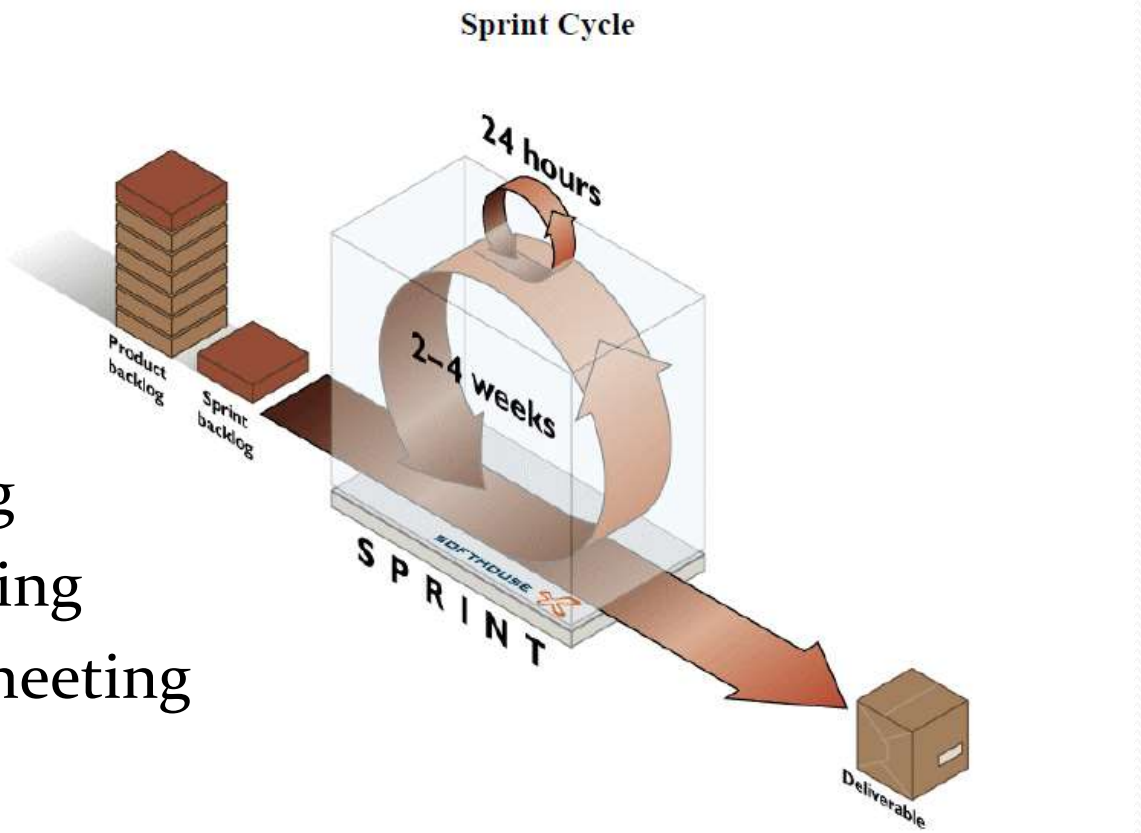


Figure 1. Scrum

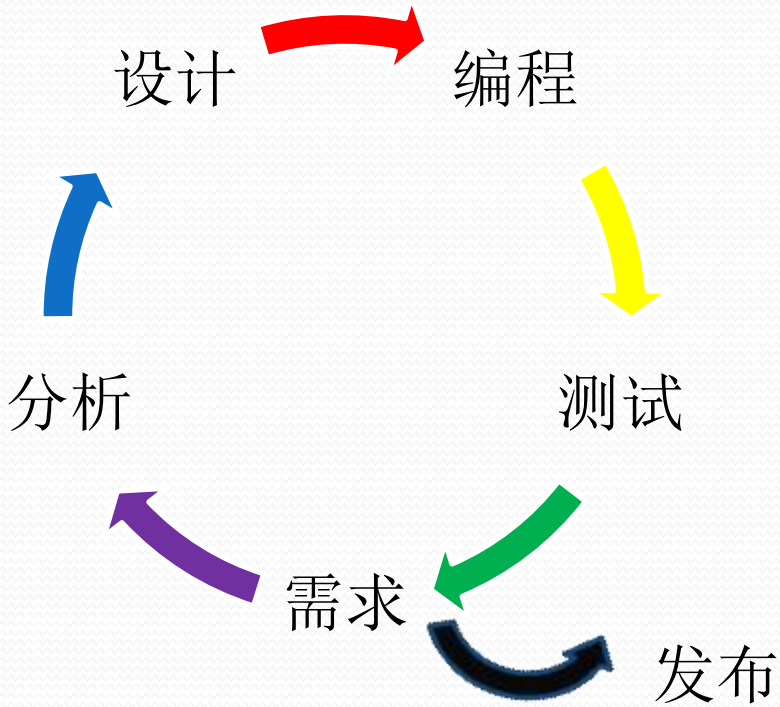
Scrum的主要概念

① sprint
② backlog
③ time-box
④ sprint backlog
⑤ daily scrum meeting
⑥ sprint review meeting
⑦ sprint planning meeting
⑧ sprint retrospective meeting

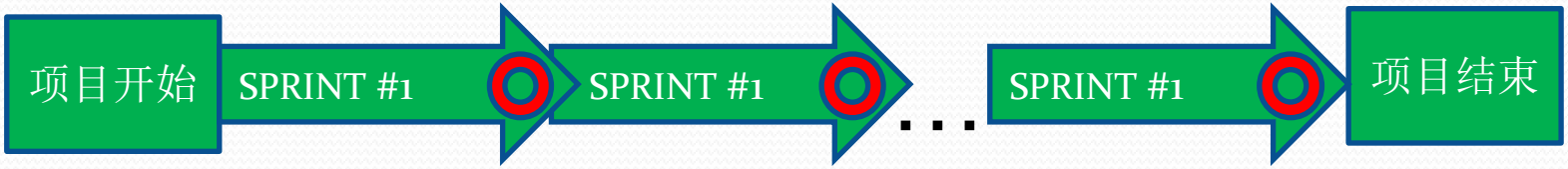


Sprint

- 一次迭代开发的时间周期，一般最多以30天为一个周期。
- 在这段时间内，开发团队需要完成一个制定的backlog，并且最终成果是一个增量的、可以交付的产品。



Sprint细节



Scrum的Sprint过程

Backlog

- 可以预知的所有任务，包括功能性的和非功能性的所有任务。
- 按优先级排列的需求列表，在顶部的条目比底部的条目更加具体（细精度）。
- 对所有的相关干系人都可见，任何相关干系人都可以增加条目，产品负责人需要时常重新排序
- 在Backlog改进会议中维护。
- 经常以“user story”的形式记录。
- 通过描述验收标准来防止技术债务。

Backlog示例

产品 BACKLOG（示例）					
ID	Name	Imp	Est	How to demo	Notes
1	存款	30	5	登录，打开存款界面，存入 10 欧元，转到我的账户余额界面，检查我的余额增加了 10 欧元。	需要 UML 顺序图。目前不需要考虑加密的问题。
2	查看自己的交易明细	10	8	登录，点击“交易”，存入一笔款项。返回交易页面，看到新的存款显示在页面上。	使用分页技术避免大规模的数据库查询。和查看用户列表的设计相似。



Figure 5. Visual Management - Sprint Backlog tasks on the wall

				New Estimates of Effort Remaining as of Day...					
Product Backlog Item	Sprint Task	Volunteer	Initial Estimate of Effort	1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart	modify database		5						
	create webpage (UI)		8						
	create webpage (Javascript logic)		13						
	write automated acceptance tests		13						
	update buyer help webpage		3						
	. . .								
Improve transaction processing performance	merge DCP code and complete layer-level tests		5						
	complete machine order for pRank		8						
	change DCP and reader to use pRank http API		13						

Figure 4. Sprint Backlog

Sprint Backlog

- 一个sprint周期内需要完成的任务，详细说明“如何”实现产品条目中的“做什么”，大概包含少于或等于一天的工作。
- 团队和产品负责人在Sprint计划会议中共同协商对backlog条目做出的承诺。
- 团队在Sprint计划会议中创建初始任务，在Sprint执行过程中不断修改。
- 必须对团队可见，在Scrum每日例会中作为参考。
- 每天以小时为单位重新评估剩下的工作量，团队成员自愿摘取任务并完成它，但是成员之间必须协调，因为对承诺负责的是整个团队。

Sprint Backlog

承诺的Backlog条目	将要做	正在做	完成
产品Backlog 条目 #1	Sprint 任务 Sprint 任务 Sprint 任务	Sprint 任务 Sprint 任务	Sprint 任务 Sprint 任务
产品Backlog 条目 #2	Sprint 任务 Sprint 任务 Sprint 任务 Sprint 任务	Sprint 任务	
产品Backlog 条目 #3	Sprint 任务 Sprint 任务 Sprint 任务		

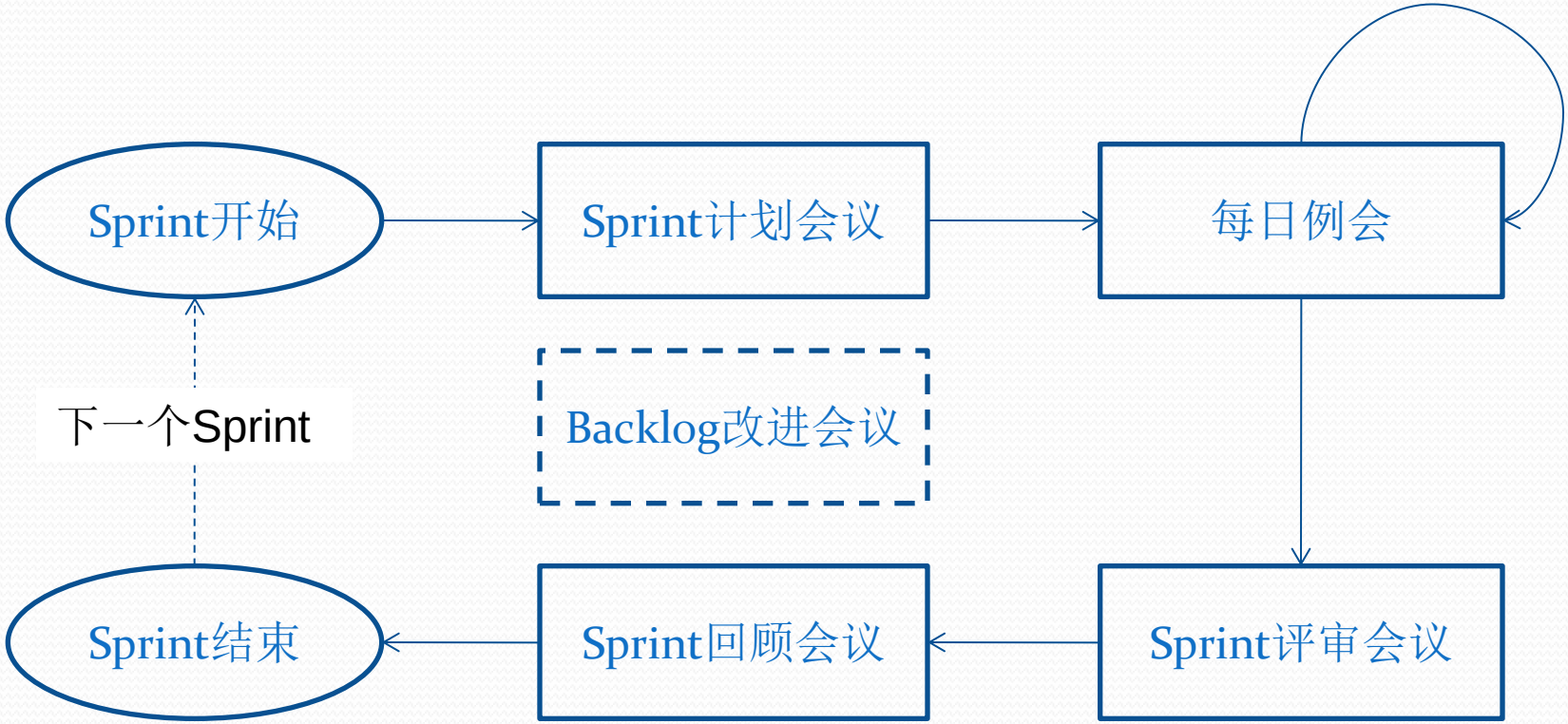
Sprint中需要完成的非编程任务：

- 搭建测试环境。
- 明确需求。
- 与运营部门讨论部署的操作细节。
- 编写部署文档（版本说明，RFC，或任何在你们组织中要写的东西）。
- 和外界的资源进行联系（例如GUI设计师）。
- 改进构建脚本。
- 将故事进一步拆分成任务。
- 标识出来自开发人员的问题，并帮助解决这些问题。

time-box

- 一个用于开会时间段。
- 通常会议的time-box：
  - sprint Planning Meeting，8小时
  - daily scrum meeting，15分钟
  - sprint Review meeting， 4 小时
  - sprint retrospective meeting，3小时

SCRUM会议



- 所有的Scrum会议都由ScrumMaster来协调，但他在会议中并有决定权力。

Sprint Planning Meeting

- 在启动每个sprint前召开的会议，会期为一个工作日（8小时）。
- Sprint计划会议不应超过Sprint所有时间的5%。在会议结束时，会产生这个Sprint的Sprint Backlog。

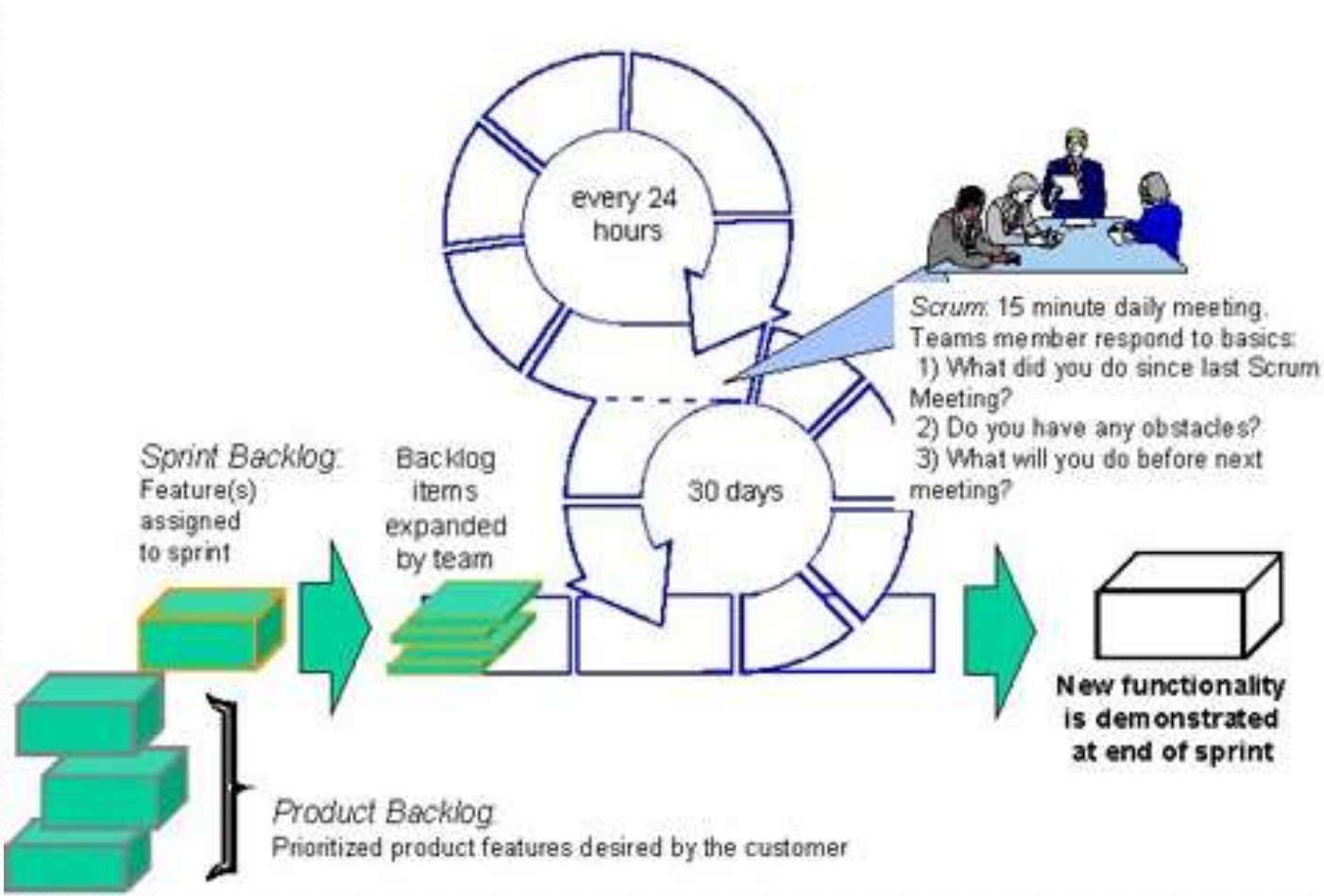
（续）

- 第一部分：
  - 在每个Sprint的开始， Product Owner和团队商讨在这个Sprint中将会做哪些产品的Backlog的条目。
  - 确定好这个Product Backlog的任务优先级。
  - Product Owner负责声明哪些条目对商业是最重的。
  - 团队负责只承诺他们在不增加技术债务的前提下能完成的工作。
- 第二部分：
  - ScrumMaster选中的Product backlog的条目分解为初始的Sprint任务，然后对将要做的工作做出最后的承诺。
  - 估算这些模块的工作量，以小时计算。
  - Product Owner并不需要全程参与第二部分的会议。

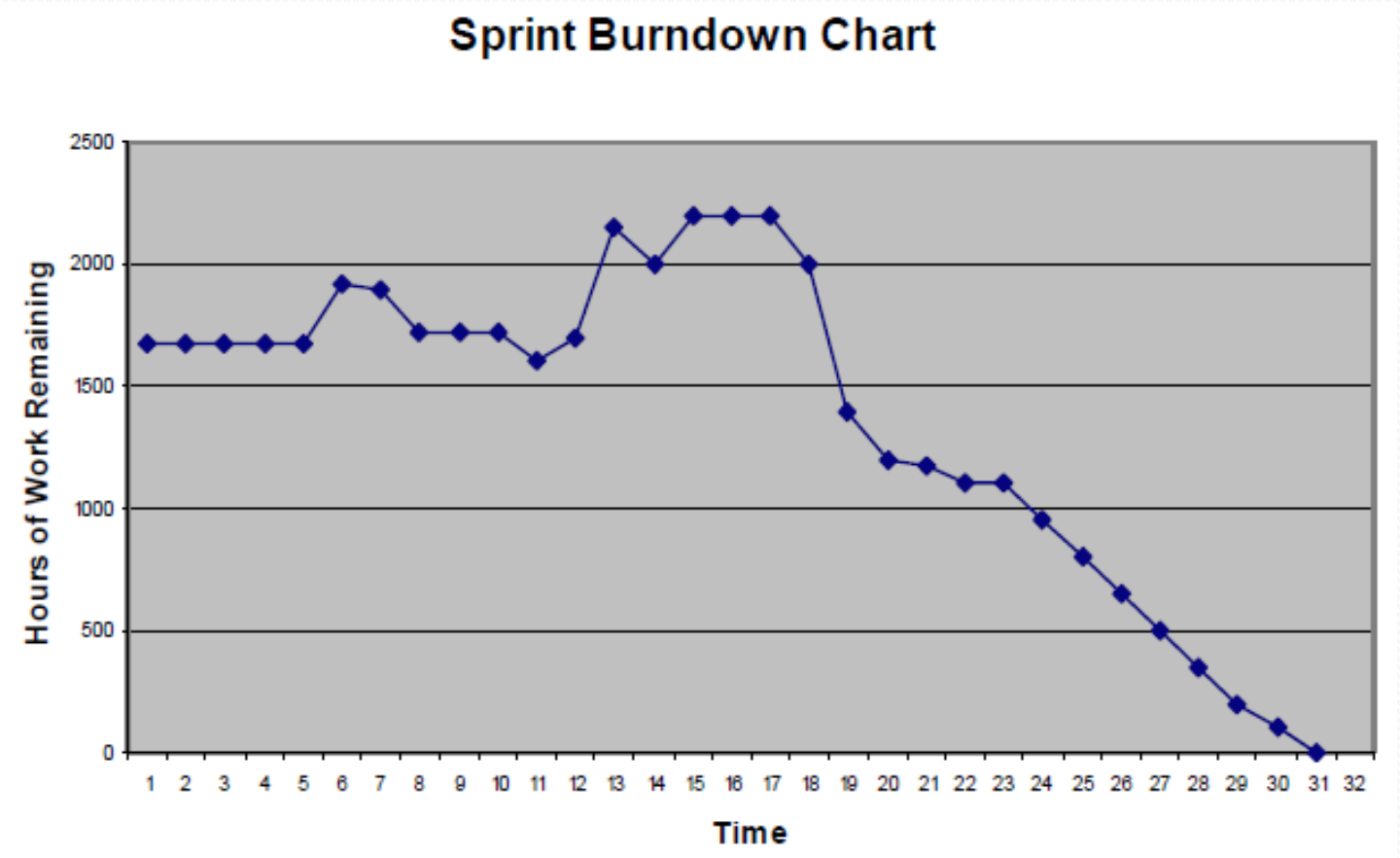
Daily Scrum meeting

- 每天同一时间同一地点，Scrum开发团队花15分钟时间互相通报工作。不是向老板汇报工作。
- 每个团队成员通报三个项目：今天完成了什么？ 是否遇到了障碍？ 即将要做什么？
- 通常情况下，团队会检查当前的Sprint Task、Sprint Burndown Chart和障碍列表。
- product owner并不需要参加每日例会。甚至他的参加会影响团队的自我管理。
- 站着开例会是一个好习惯，这会增加紧迫感。

Daily Scrum meeting时机



Sprint Burndown Chart



Sprint Review Meeting

- 在每个sprint结束时，团队向product owner和相关干系人演示他们所完成的产品增量，会议一般为 4 小时。
- product owner根据之前的商讨协议，声明哪些承诺的条目可以认为“完成”。
- 没有完成的条目将被退回到产品backlog，作为下一个未来sprint的候选项。
- 从相关干系人那里得到的反馈也会被转成新的产品backlog条目。
- sprint评审会有浮现的需求，做需求变更。

Sprint retrospective meeting

- 对刚结束的sprint进行总结。
- 参加会议的人员为开发团队的内部人员。
- 一般该会议为 3 小时。
- 会议给团队提供一个观察并调整的流程机制。
- 通常的话题：
  - 哪些做很好
  - 哪些需要改进
  - 我们学到了什么
  - 什么仍然是我们的难题
  - 我们应该采取哪些行动

Scrum中角色

- Product Owner（产品负责人）
  - 熟悉产品的远景和发展方向，负责使产品的研发投入获得最大的投资回报。
  - 需要经常重新排列产品需求的优先级，并且修正对发布计划的期望，他是需求的最终决定者。
  - 在每个Sprint评审会议中，他可能判定哪些已经完成。
  - 与产品的客户和终端用户紧密合作。

- Scrum Development Team（Scrum开发团队）
  - 通常由5~9名成员组成；
  - 是跨职能的，其中可以包含某方面的专家，具有技能的工程师。
  - 团队对他们从Sprint计划会议到Sprint评审会议之间的所有承诺负责。而且，在对于如何实践承诺方面，他们是自治的。
  - 如果一个Scrum开发团队能坐在同一个房间工作，他们将更容易成功。

- ScrumMaster
  - 他协助Scrum流程实施，保证工件可见。
  - 他推动团队的自我管理，帮助铲除障碍，保证团队不受干扰，宣扬并鼓励工作实践的改善。
  - 他并没有掌控团队的权利，他不做商业决策或者技术决策。
  - 他不替团队做工作以上的以及其它的任何承诺。他是带路者，但不是管理者。

相关实践

- 持续集成
- 测试驱动开发
- 自动测试
- 持续测试
- 持续重构
- 结对编程

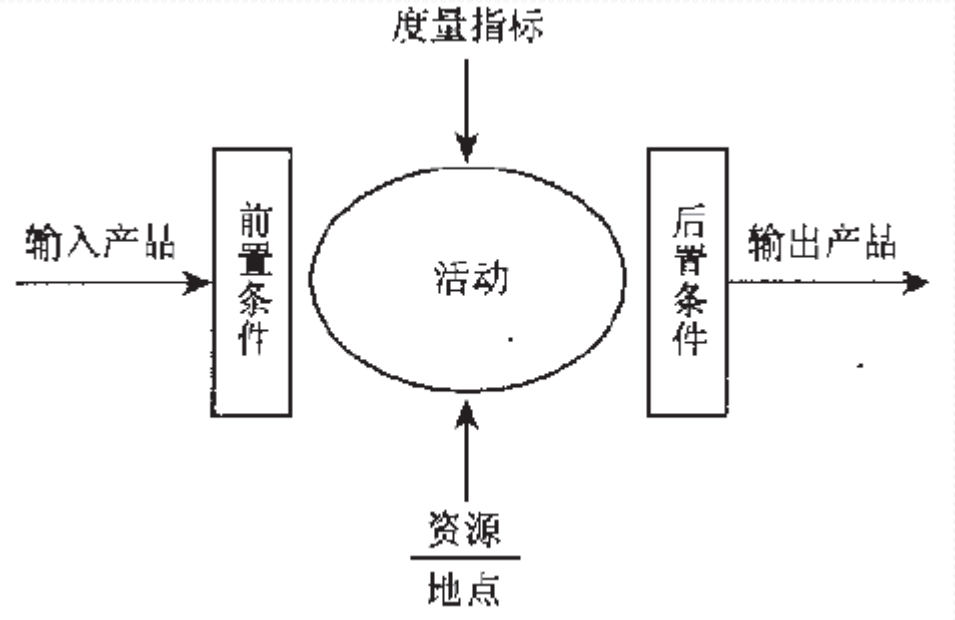
SCRUM小结

- 包含一个或多个“自我管理”并且“跨职能”的团队。每个团队由5~9名成员组成。
- 使用time-box的迭代，每个迭代周期被称为sprint，长度通常为2~4周。
- 团队在每个sprint都会全力开发一个潜在可交付的产品增量。因此，在每个sprint结束时，产品都会增加有高商业价值的新功能 。
- 通常以 30 天为一个阶段，由客户提供新产品的需求规格开始，开发团队必须尽力于 30 天后交付成果。
- 团队每天用 15 分钟开会检查每个成员的进度与计划，了解所遭遇的困难并设法排除。

第7讲 敏捷方法

5. 敏捷开发对软件质量的影响

敏捷过程定义



敏捷方法的假设

① 需求变化和变化难以预测。

② 软件设计与软件构造之间存在间隔，设计的正确性难以预测。

③ 软件的分析、设计、构造和测试难以预测。

敏捷开发要避免的过程设计的几个常见错误：

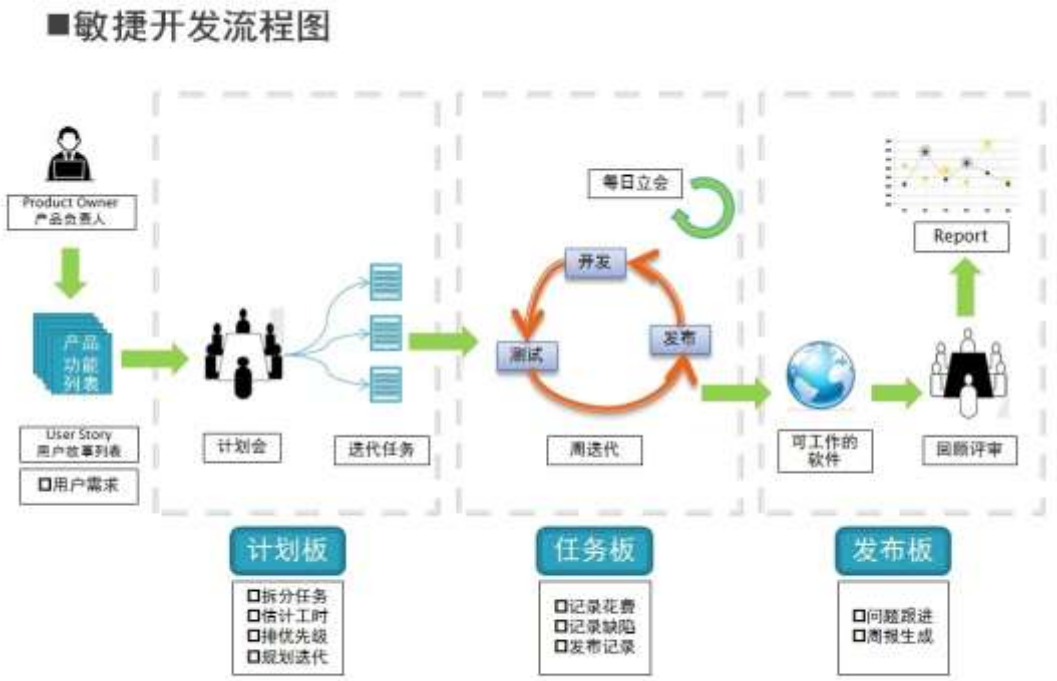
(1)增加不必要的“必须完成”(“Should do” is really should?)；

(2)对所有的项目使用同一种过程；

(3)过于沉重；

(4)没有弹性；

(5)没有经过实践检验。



敏捷开发中常见问题

- 在软件开发中，人的因素要远远大于过程和技术。人是有缺陷的：

  (1)容易犯错误，因此必须在错误扩散之前找到并改正错误；

  (2)当觉得可能失去较多的时候，不愿意冒险；

  (3)重新构造而不愿意重复使用已有的东西；

  (4)难于坚持一个习惯。

- 敏捷开发方法要避免的过程设计的几个常见错误：

  (1)对所有的项目使用同一种过程；

  (2)没有弹性；

  (3)过于沉重；

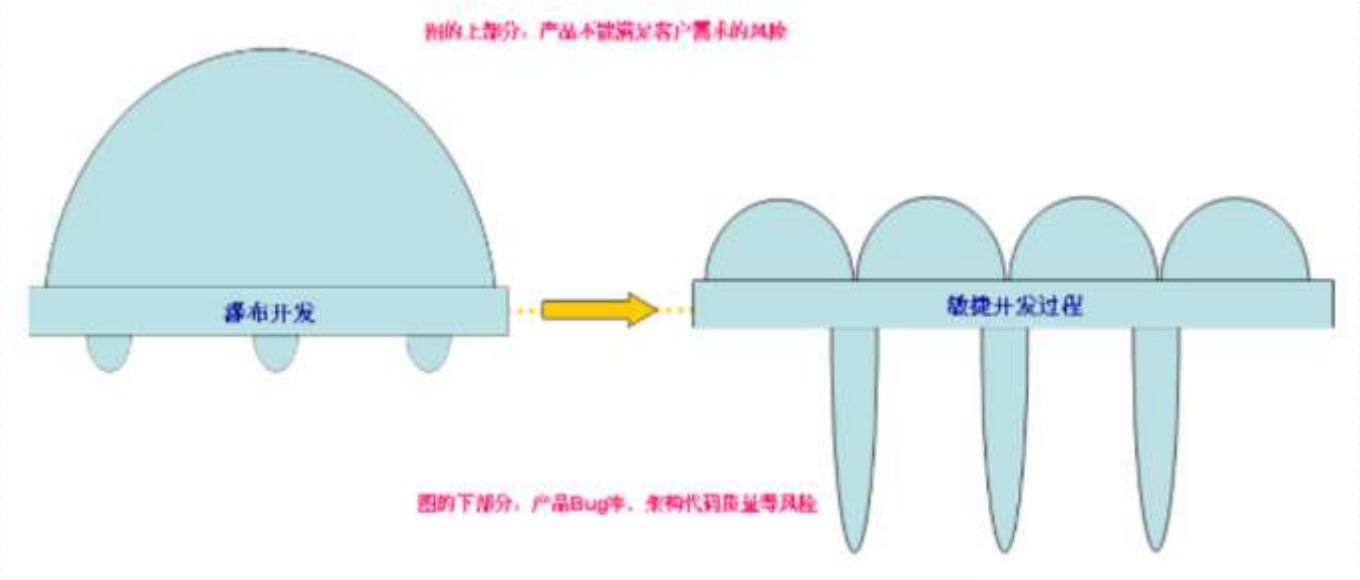
  (4)增加不必要的“必须完成”(“Should do” is really should?)；

  (5)没有经过实践检验。

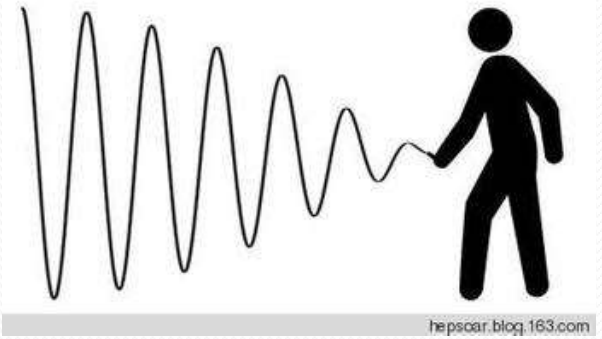
敏捷开发中的如下特性：

- 需求变化更频繁
- 轻详细设计，没有那么多的架构和设计的时间
- 反对冗余繁重的文档
- 反对冗长会议和电话会议
- 赞同代码就是最好设计和文档（Code is design and document），并通过重构自然呈现架构和设计

敏捷过程带来的影响



- 敏捷开发对“正确性的软件”的影响
- 敏捷开发对“软件运行正确”的影响



长鞭效应（bullwhip effect)

敏捷开发中人的问题

- 在软件开发中，人的因素要远远大于过程和技术，人是有缺陷的：

  (1)容易犯错误，因此必须在错误扩散之前找到并改正错误；

  (2)当觉得可能失去较多的时候，不愿意冒险；

  (3)重新构造而不愿意重复使用已有的东西；

  (4)难于坚持一个习惯。

- 敏捷方法中人的因素（Human Factors）：
  - Competence 能力
  - Common focus 共同的焦点
  - Collaboration 协作
  - Decision-making ability 决策能力
  - Fuzzy probe-solving ability 模糊探测能力
  - Mutual trust and respect 相互信任和尊重
  - Self-organization 自组织

Persona方法论

- Persona指的是角色，也就是基于角色的方法论，强调一切从角色出发思考问题。
- 由于每个人思考的角度不同，客户思考问题的角度和开发团队的思考角度也不一样，甚至不同客户之间思考问题的角度也不样。
- 团队在设计和开发软件时候，容易陷入自己的思维角度，开发出不符合客户思维角度的软件，而不符合客户的需求。更严重的是很多团队陷入这样的惯性中，而根本不知道错误在哪里。



敏捷需要“good people”

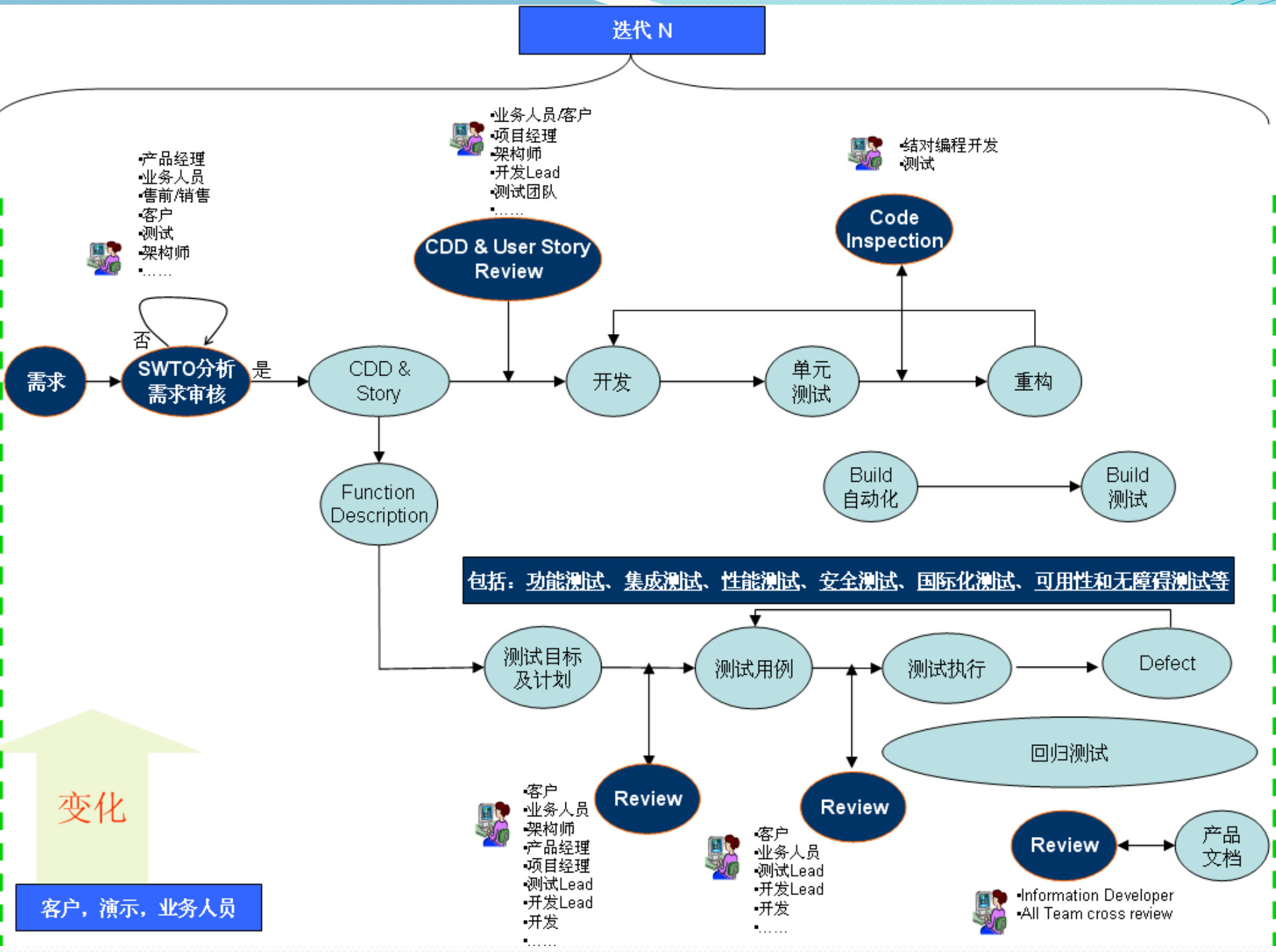
A certain percentage of experienced people are needed for a successful Agile project. There was some consensus that 25%-33% of the project personnel must be “competent and experienced.” “Competent” in this context means:

- Possess real-world experience in the technology domain
- Have built similar systems in the past
- Possess good people & communication skills.

[Lindvall M , Basili V , Boehm B , et al. Empirical Findings in Agile Methods[M]// Extreme Programming and Agile Methods — XP/Agile Universe 2002. Springer Berlin Heidelberg, 2002.]

Agile methods derive much of their agility by relying on the tacit knowledge embodied in the team, rather than writing the knowledge down in plans.

[Boehm B W . Get Ready for Agile Methods, With Care[J]. Computer, 2002, 35(1):64-69.]


迭代 N
•产品经理
•业务人员
•售前/销售
•客户
•测试
•架构师
•……
•业务人员/客户
•项目经理
•架构师
•开发Lead
•测试团队
•……
•结对编程开发
•测试
CDD & User Story Review
Code Inspection
否
需求
SWTO分析 需求审核
是
CDD & Story
开发
单元 测试
重构
Function Description
Build 自动化
Build 测试
包括：功能测试、集成测试、性能测试、安全测试、国际化测试、可用性和无障碍测试等
测试目标 及计划
测试用例
测试执行
Defect
回归测试
Review
Review
Review
产品 文档
•客户
•业务人员
•架构师
•产品经理
•项目经理
•测试Lead
•开发Lead
•开发
•……
•客户
•业务人员
•测试Lead
•开发Lead
•开发
•……
•Information Developer
•All Team cross review
变化
客户，演示，业务人员

第7讲 敏捷方法

6. 敏捷过程与传统方法

计划驱动的开发方法

- 计划驱动开发起源于系统工程和质量规范，建立系统工程的原则，协调大量需要精确协同工作的组件。
- 通过从需求到已完成的代码等一系列代表物来推动软件开发的过程，计划驱动开发非常精确地依赖于明确的步骤。

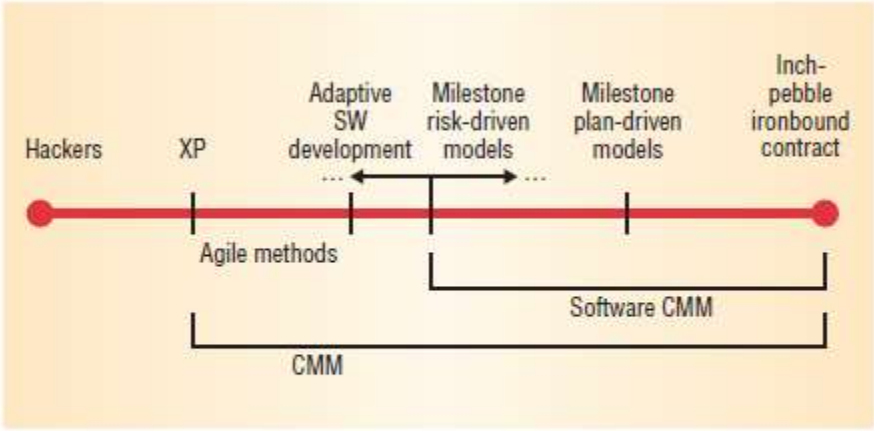


Figure 1. The planning spectrum. Unplanned and undisciplined hacking occupies the extreme left, while micromanaged milestone planning, also known as inch-pebble planning, occupies the extreme right.

- 计划驱动开发的关键是过程的定义和管理，与过程改进联系在一起，强势在于标准化所带来的可比较性和可重复性。过程需要进行定义、标准化并需要逐步改进以提供控制。管理其操作所需的数据。

敏捷方法与计划驱动比较

要素	敏捷方法	计划驱动方法
规模	适合小型产品和团队	适合大型产品和团队
危险	没有经过安全关键性产品的考验。简单的设计和缺乏文档有潜在问题	适合应对高安全性的产品。
动态性	简单设计和持续重构非常适于高度动态的环境，但对于高度稳定环境会导致潜在的返工	详细设计和庞大的预先设计非常适合于高度稳定的环境，但是对于高度动态的环境导致返工。
人员	整个过程都需要高素质人员和专家参与。	项目初期需要高素质人员和专家，项目平稳进行期间，对人员的素质要求可以降低。
文化	更高的自由度，使人们感到舒适、有权力。 靠混沌繁荣。	清晰的政策和规程定义了人们的角色，使人们感到舒适、有权利。 靠秩序繁荣。

Agile and plan-driven methods

[Boehm B W . Get Ready for Agile Methods, With Care[J]. Computer, 2002, 35(1):64-69.]

Table 1. Home ground for agile and plan-driven methods.

Home-ground area	Agile methods	Plan-driven methods
Developers	Agile, knowledgeable, collocated, and collaborative	Plan-oriented; adequate skills; access to external knowledge
Customers	Dedicated, knowledgeable, collocated, collaborative, representative, and empowered	Access to knowledgeable, collaborative, representative, and empowered customers
Requirements	Largely emergent; rapid change	Knowable early; largely stable
Architecture	Designed for current requirements	Designed for current and foreseeable requirements
Refactoring	Inexpensive	Expensive
Size	Smaller teams and products	Larger teams and products
Primary objective	Rapid value	High assurance

Both agile and plan-driven methods have a home ground of project characteristics in which each clearly works best, and where the other will have difficulties. Hybrid approaches that combine both methods are feasible and necessary for projects that combine a mix of agile and plan-driven home-ground characteristics.

敏捷软件开发与计划驱动开发的对比

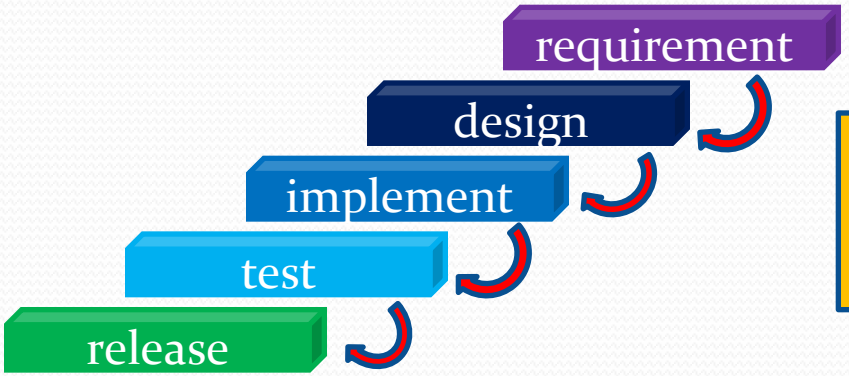
- 应用：
  - 项目的主要目标、项目环境和应用环境。
- 管理：
  - 客户关系、计划和控制，以及项目沟通。
- 技术：
  - 需求定义、开发和测试的方法。
- 人员：
  - 客户特征、开发人员的特征，以及组织的文化。

80/20原理启示

- 20世纪，意大利经济学家兼社会学家帕雷托提出一种“重要的少数与琐碎的多数”原理。这个原理经过多年的演化，已变成当今管理学界所熟知的“80/20原理”。
- “80/20原理”的重要启示：
  - 在项目工作中也应将时间花在重要少数问题上。
  - 开发系统的过程中，根据事情的重要程度安排其的优先次序。
  - 试图在第一个版本中一劳永逸地解决所有问题是行不通的。

观点：敏捷开发与传统模型

“命令-控制”模型



“领导-协作”模型



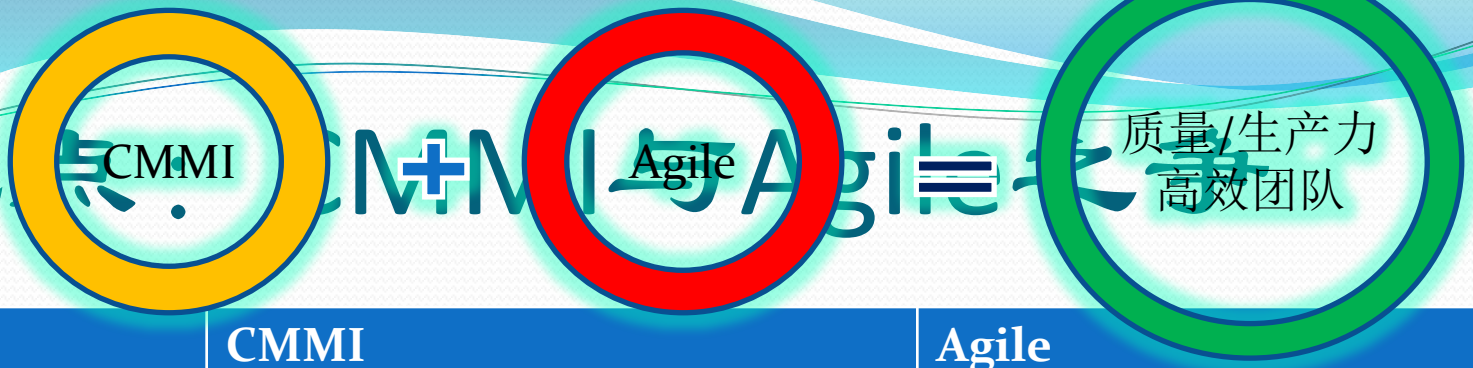
- 强调流程和管理
- 不能很好适应变化
- 重视计划，“憎恨”变更
- 文档化要求高
- 重视外部力量对项目的控制
- 管理成本一般较高
- 领导拥有绝对权威
- 研发周期一般较长

- 强调团队协作
- 针对变化而生，容变！
- 灵活的计划，短周期
- 降低文档化要求
- 强调代码共享
- 强调项目组内部自发管理
- 管理成本较低，对团队要求高
- 项目组成更受关注

观点： 对敏捷方法与传统方法意见

- 单纯的敏捷方法无法解决所有问题
  - 存在较多的前提条件，对企业文件、团队、项目类型、项目研发方式均有要求
  - 不适合自研+外包结合的项目
- 传统方法具备一些可取的特征
  - 要求显示的管理规则
  - 某些控制手段和要求适合于外包管理
  - 审计、测量等工作需保留

观点：CMMI与Agile之争



维度	CMMI	Agile
早期客户	大型项目、复杂系统、使命关键系统	小项目、简单应用、灵活多变的系统
假想市场/用户	成熟市场、关注过程创新	新兴市场，多变的市场环境
关注点	组织过程改进 关注回答该做什么，关注过程绩效	项目级改进 关注项目具体怎么做，交付客户价值
文化	适用于高信任和低信任环境 强调管理者和管理的作用 关注过程，有限地关注人	倡导高信任的环境 强调管理者的教练作用 关注团队和个人，强调激励和自组织
核心价值观	关注统一、标准的过程Agile过程改进，关注可预测性和质量	关注通过团队协作，快速响应，通过可工作的软件，持续交付价值

敏捷与CMMI比较

- 相同的理念：

  都承认变化，要求适应变化。

- 解决问题方法的差异：
  - 敏捷注重适应性（adap
  - CMMI注重制度化（ins

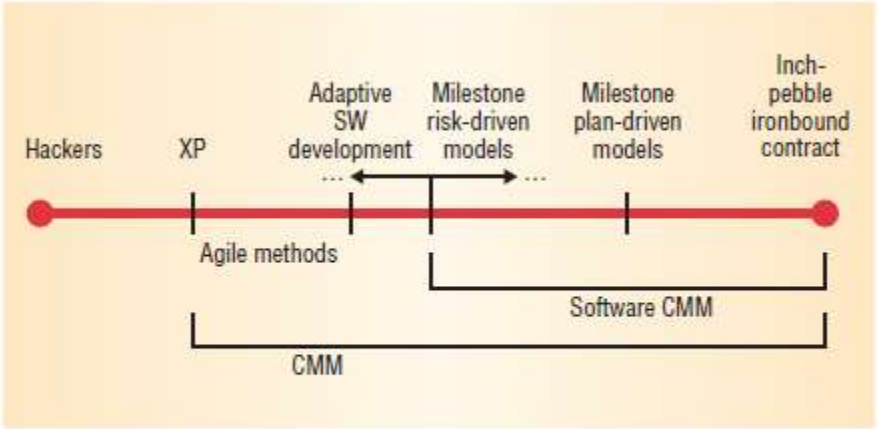


Figure 1. The planning spectrum. Unplanned and undisciplined hacking occupies the extreme left, while micromanaged milestone planning, also known as inch-pebble planning, occupies the extreme right.

总结

- 敏捷方法重人和技术，轻过程。
- 现有CMMI不适用，敏捷注重个体的能力和技术，CMMI注重团队能力。
- 需求驱动，计划指导的方法不一定是最好的方法。
- 敏捷方法不能替代传统的方法。

敏捷过程的局限性

1. 缺乏对分布式开发环境的支持
2. 缺乏对转包合同的支持
3. 缺乏对创建可复用制品的支持
4. 缺乏对大型团队开发的支持
5. 缺乏对开发有严格安全性要求的软件的支持
6. 缺乏对开发大型、复杂的软件的支持

课堂讨论：

① 举例说明敏捷方法的开发过程。

② 如何理解敏捷的理念？

③ 给敏捷开发方法下定义

④ 敏捷方法与传统方法各自的优缺点。

⑤ 敏捷方法开发中可能出现的什么问题？如何应对这些问题？